

PERFORMANCE

Testing

& Apache JMeter

- ✓ Why, When & What of Performance Testing
- ✓ SDLC Integration & Performance Life Cycle
 - ✓ NFR Gathering & Test Strategy
 - ✓ Performance Test Types & Roles
 - ✓ JMeter Installation & Setup
- ✓ Script Recording: Proxy, BlazeMeter & HAR Conversion
- ✓ Playback, Validation & Complete User Journey Activity

Comprehensive Study Material with Practical Examples

Table of Contents

1. Introduction to Performance Testing

- Why Performance Testing?
- When is it Required?
- What Should Be Tested?
- Objectives & Definitions
- SDLC & Performance Testing Life Cycle
- Architecture & Environments
- Performance Test Types
- Roles & Responsibilities

2. NFR Gathering & Strategy

- NFR / Performance Testing Requirements
- Introduction to Performance Test Strategy
- Complete Performance Test Strategy Document
- Review Examples

3. JMeter Setup

- JMeter Installation
- Classpath Setup for Recording
- Increase System TCP/IP Ports
- Increase JMeter Heap Memory

4. JMeter Recording

- Proxy Setup
- Introduction to JMeter Recording
- Script Recording with JMeter
- Script Recording using BlazeMeter
- HAR to JMX Conversion
- Playback and Validation of Request/Response
- Activity: Record Complete User Journey

SECTION 01

Introduction to Performance Testing

Fundamentals, Life Cycle, Architecture & Test Types

Why Performance Testing?

Performance testing verifies that a system behaves correctly and efficiently under expected and peak load conditions. It answers the critical business question: *"Will the system hold up when real users hit it?"*

Real-world failures demonstrate the cost of neglecting performance testing:

- **Amazon:** A 100ms delay costs ~1% in sales revenue.
- **Google:** An extra 500ms in search page generation drops traffic by 20%.
- **E-commerce Black Friday:** Unprepared sites crash under 10x normal traffic — losing millions.
- **Healthcare/Banking:** Slow systems cause regulatory violations and user trust erosion.

■ **Warning:** Performance issues found in production cost 10–100x more to fix than those found during testing. Early detection is always cheaper.

When is Performance Testing Required?

Trigger	Description
New Release	Any major release or architectural change
Infrastructure Change	Server upgrades, cloud migration, DB changes
Pre-Production	Before go-live to validate capacity
Post-Incident	After a production performance outage
Seasonal Events	Black Friday, elections, sporting events
Regulatory Requirement	SLA / SLO compliance validation
Third-party Integration	New API or service dependency added

What Should Be Tested?

Performance testing does not mean testing every single endpoint. Focus on business-critical transactions and system boundaries:

Metric	What It Measures
Response Time	Time taken by the system to respond to a user request
Throughput	Number of requests processed per second (TPS / RPS)
Resource Utilisation	CPU, Memory, Disk I/O, Network during load
Scalability	Behaviour as load increases linearly or exponentially
Bottlenecks	Slow DB queries, thread pool exhaustion, memory leaks
Error Rate	% of failed requests under load conditions

Concurrency	Behaviour with simultaneous users performing operations
-------------	---

Objectives & Definitions

Term	Definition
Response Time	Total time from request sent to response fully received by client.
Latency	Time for first byte of response to arrive (network + server processing).
Throughput (TPS)	Transactions Per Second — how many completed operations per second.
Concurrent Users	Users actively sending requests at the same moment.
Think Time	Pause time between user actions (simulates real human behaviour).
Ramp-Up Period	Time taken to reach the target number of virtual users.
SLA	Service Level Agreement — contractual performance threshold (e.g., 95% requests < 2s).
SLO	Service Level Objective — internal target (e.g., p99 latency < 500ms).
Percentile (p95, p99)	95% or 99% of requests are faster than this value.
Error Rate	Percentage of requests returning errors (4xx, 5xx, timeouts).

SDLC & Performance Testing Life Cycle

Performance testing integrates into every phase of the Software Development Life Cycle (SDLC). Identifying the right entry points ensures issues are caught early:

SDLC Phase	Performance Activity
Requirements	Gather NFRs; define SLAs and acceptance criteria
Design	Review architecture for performance bottlenecks; model capacity
Development	Component/unit-level performance benchmarks; API profiling
Testing (SIT/UAT)	Full performance test cycles: load, stress, soak
Pre-Production	Baseline establishment; capacity validation; sign-off
Production	Monitoring, APM alerts, post-deployment performance checks
Maintenance	Regression performance tests on every major change

Performance Testing Life Cycle (PTLC)

1. NFR Analysis & Workload Modelling
2. Test Environment Identification & Setup
3. Test Strategy & Plan Creation
4. Test Script Development & Review
5. Test Data Preparation
6. Test Execution (Load / Stress / Soak)
7. Performance Monitoring & Data Collection
8. Results Analysis & Bottleneck Identification
9. Reporting & Recommendations
10. Re-test after Fixes (Regression)

Architecture & Environments

A typical performance test architecture consists of load generators, the system under test (SUT), and monitoring infrastructure. The environment should mirror production as closely as possible for meaningful results.

Component	Role	Tools/Examples
Load Generator	Simulates virtual users sending requests	JMeter, k6, Gatling, Locust
Controller Node	Orchestrates distributed load test	JMeter Master, Taurus
Worker Nodes	Execute actual virtual user load	JMeter Slaves, cloud agents
System Under Test	Application servers, APIs, DB, cache	Java app, Node.js, PostgreSQL
APM / Monitoring	Captures server-side metrics	Grafana, Dynatrace, New Relic, Prometheus
Results Store	Stores test results for analysis	InfluxDB, Elasticsearch

Environment Considerations:

- **Production-like:** Same hardware spec, same OS, same middleware versions.
- **Isolated:** No shared resources with functional test environments.
- **Network:** Simulate real-world latency if testing from cloud.
- **Data Volume:** DB should have production-sized data for realistic query plans.
- **Third-party Stubs:** Mock external services to avoid rate limits during tests.

Performance Test Types

Test Type	Purpose	Load Pattern	Duration
Load Test	Validate system under expected load	Ramp up to target concurrent users	1–2 hours
Stress Test	Find breaking point beyond normal capacity	Incrementally increase beyond limits	2–4 hours
Soak / Endurance	Detect memory leaks, resource degradation	Sustained load over long period	8–72 hours
Spike Test	Simulate sudden traffic spikes	Instant jump to peak then back	30–60 min
Volume Test	Test with large data volumes in DB	Normal load, massive data sets	Varies
Scalability Test	Verify linear scaling with resources	Step-wise load increase	2–3 hours
Baseline Test	Establish reference benchmark	Single user or low fixed load	15–30 min
Smoke / Sanity	Quick check before full test run	Minimal load (1–5 users)	5–10 min

■ **Tip:** Always run a Baseline test before Load/Stress to establish a clean reference. Without a baseline, you cannot measure degradation.

Roles & Responsibilities

Role	Key Responsibilities
Performance Test Lead	Strategy, planning, stakeholder communication, sign-off
Performance Test Engineer	Script development, execution, analysis, reporting
DevOps / Infra Engineer	Environment provisioning, monitoring setup, network config
Application Developer	Code profiling, bottleneck fixes, build support
Business Analyst	NFR definition, workload model, acceptance criteria
Test Manager	Resource planning, risk management, governance
Architect	Architecture review, capacity modelling, design input

SECTION 02

NFR Gathering & Strategy

Requirements • Strategy Document • Review Examples

What are NFRs?

Non-Functional Requirements (NFRs) define *how well* the system performs its functions, as opposed to functional requirements that define *what* the system does. For performance testing, NFRs are the acceptance criteria your test must validate.

NFR Categories:

Category	Example NFR
Response Time	Max acceptable latency per transaction (e.g., Login < 2s at p95)
Throughput	Minimum TPS the system must sustain under load
Concurrent Users	Number of simultaneous active users the system must support
Error Rate	Maximum acceptable error rate (e.g., < 0.1% errors under load)
Resource Limits	Max CPU %, Memory %, Disk I/O during load
Availability	System uptime requirement (e.g., 99.9% — ~8.7 hrs downtime/year)
Scalability	System scales linearly up to X times current capacity
Recovery Time	System recovers within Y seconds after a spike

NFR Gathering Process

NFRs must be gathered from multiple stakeholders. They cannot be assumed or invented by the testing team. The process:

Step 1: Stakeholder Interviews

Meet with Business Owners, Architects, and Operations to understand SLAs and peak load expectations.

Step 2: Analyse Production Data

Review current production analytics — peak concurrent users, top transactions, average response times, error rates.

Step 3: Business Workload Model

Map user journeys to percentages (e.g., 40% browse, 30% search, 20% checkout, 10% account).

Step 4: Define Acceptance Criteria

Convert stakeholder inputs into testable, measurable criteria with pass/fail thresholds.

Step 5: Sign-Off

Get written agreement from all stakeholders before test execution begins.

Sample NFR Document

Transaction	Concurrent Users	Avg Response (p50)	p95 Response	Error Rate
Homepage Load	1000	< 1.0s	< 2.0s	< 0.1%
User Login	500	< 1.5s	< 3.0s	< 0.1%
Product Search	800	< 2.0s	< 4.0s	< 0.5%
Add to Cart	400	< 1.0s	< 2.0s	< 0.1%
Checkout / Payment	200	< 3.0s	< 5.0s	< 0.1%
Order History	300	< 2.0s	< 3.5s	< 0.2%

Performance Test Strategy

The Performance Test Strategy is a high-level document that defines the approach, scope, tools, environment, entry/exit criteria, and risks for the entire performance testing engagement.

Strategy Document Structure:

Section	Content
1. Executive Summary	High-level overview of purpose, scope, and key decisions
2. Objectives	What will be validated (response time, throughput, stability)
3. Scope	In-scope transactions, systems, browsers, devices
4. Out of Scope	Explicit exclusions with justification
5. Test Types	Which test types (load, stress, soak) and when they run
6. Workload Model	Concurrent users, TPS targets per transaction
7. Test Environment	Spec of servers, network config, data volumes
8. Tools	JMeter version, monitoring tools, reporting tools
9. Entry Criteria	What must be true before testing starts
10. Exit Criteria	What must be true to declare testing complete
11. Deliverables	Scripts, reports, dashboards, recommendations
12. Risks & Mitigation	Known risks and contingency plans
13. Schedule	Timeline for each test cycle
14. Sign-off	Stakeholder approval section

Entry & Exit Criteria Examples

Entry Criteria:

- Performance test environment is provisioned and stable
- All functional test scripts are peer-reviewed and signed off
- Test data is prepared and loaded into the environment
- Monitoring tools (APM, Grafana) are configured and verified
- NFRs and acceptance criteria are reviewed and approved
- Application build is deployed and smoke tested (no P1 defects)

Exit Criteria:

- All planned test scenarios have been executed at least once
- All P1/P2 performance defects are resolved and re-tested

- NFR thresholds are met or formally accepted by stakeholders
- Performance test report is reviewed and approved
- Root cause analysis is documented for any failed criteria

SECTION 03

JMeter Setup

Installation • Classpath • TCP/IP Ports • Heap Memory

What is Apache JMeter?

Apache JMeter is an open-source, Java-based load testing tool. It simulates multiple virtual users sending requests to a system and measures how it responds. JMeter supports HTTP, HTTPS, JDBC, FTP, SOAP, REST, WebSocket, and more.

- **Free & Open Source:** Apache License 2.0
- **Cross-platform:** Windows, Linux, macOS
- **Extensible:** Rich plugin ecosystem via JMeter Plugins Manager
- **Distributed:** Master-slave architecture for large-scale load
- **GUI & Non-GUI:** GUI for script development, CLI for execution

JMeter Installation

Prerequisites:

- **Java JDK 8 or later** must be installed (JDK 11+ recommended)
- **JAVA_HOME** environment variable must be set correctly

Step-by-Step Installation:

Step	Action	Details
Step 1	Download JDK	Go to https://adoptium.net and download Eclipse Temurin JDK 17 or 21
Step 2	Set JAVA_HOME	Windows: System > Advanced > Env Variables > New: JAVA_HOME = C:\Program Files\Java\jdk-17\bin
Step 3	Add to PATH	Append %JAVA_HOME%\bin to the PATH variable
Step 4	Verify Java	Open terminal: java -version (should show 17.x or later)
Step 5	Download JMeter	Go to https://jmeter.apache.org/download_jmeter.cgi and download binary ZIP
Step 6	Extract ZIP	Extract to C:\JMeter (Windows) or /opt/jmeter (Linux/Mac)
Step 7	Launch JMeter	Windows: Double-click bin\jmeter.bat Linux/Mac: ./bin/jmeter.sh

```
# Linux / macOS installation commands

# 1. Download JMeter

wget https://downloads.apache.org/jmeter/binaries/apache-jmeter-5.6.3.tgz

# 2. Extract

tar -xzf apache-jmeter-5.6.3.tgz

mv apache-jmeter-5.6.3 /opt/jmeter

# 3. Add to PATH in ~/.bashrc or ~/.zshrc

export JMETER_HOME=/opt/jmeter

export PATH=$JMETER_HOME/bin:$PATH

# 4. Reload profile and launch

source ~/.bashrc

jmeter
```

Classpath Setup for Recording

To enable HTTPS recording, JMeter needs additional JAR files and SSL certificates added to its classpath. The JMeter proxy captures HTTPS traffic by acting as a man-in-the-middle using its own certificate.

Required JARs for Recording:

- **bcprov-jdk15on-xxx.jar** — Bouncy Castle cryptography provider for SSL
- **bcpkix-jdk15on-xxx.jar** — Bouncy Castle PKIX for certificate handling
- Place both JARs in: **JMETER_HOME/lib/**

```
# Download Bouncy Castle JARs

# Visit: https://www.bouncycastle.org/latest_releases.html

# Copy to JMeter lib directory

cp bcprov-jdk15on-1.70.jar /opt/jmeter/lib/

cp bcpkix-jdk15on-1.70.jar /opt/jmeter/lib/

# JMeter SSL Certificate (auto-generated on first proxy run)

# Location: JMETER_HOME/bin/ApacheJMeterTemporaryRootCA.crt

# Import this cert into your browser to record HTTPS traffic

# Windows: Double-click .crt > Install Certificate > Trusted Root CAs

# Firefox: Settings > Certificates > Import

# Chrome: Uses OS certificate store (import via Windows Cert Manager)
```

Increase System TCP/IP Ports

During load tests, JMeter opens thousands of TCP connections. Default OS settings limit the number of available ports and socket TIME_WAIT duration, causing '**Connection refused**' errors under high load. These must be tuned before testing.

Windows TCP/IP Tuning:

```
# Open Registry Editor (regedit) and navigate to:

# HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters

# 1. Increase ephemeral port range (default: 1024-5000)

# Add DWORD: MaxUserPort = 65534

# 2. Reduce TIME_WAIT period (default: 240 seconds)

# Add DWORD: TcpTimedWaitDelay = 30

# 3. Enable port reuse

# Add DWORD: TcpNumConnections = 65000

# Apply via PowerShell (run as Administrator):

netsh int ipv4 set dynamicport tcp start=1024 num=64511

netsh int ipv4 show dynamicport tcp

# Restart required for registry changes
```

Linux TCP/IP Tuning:

```
# Add to /etc/sysctl.conf for persistence

# Increase local port range

net.ipv4.ip_local_port_range = 1024 65535

# Reduce TIME_WAIT

net.ipv4.tcp_tw_reuse = 1

net.ipv4.tcp_fin_timeout = 15

# Increase max open files

fs.file-max = 1000000

# Apply immediately (no restart needed)

sudo sysctl -p

# Verify settings

sysctl net.ipv4.ip_local_port_range
```

Increase JMeter Heap Memory

JMeter runs on the JVM. Default heap memory is often insufficient for large tests with thousands of virtual users or heavy listeners. Insufficient heap causes **OutOfMemoryError** and test instability.

Method 1 – Edit jmeter.bat / jmeter.sh:

```
# Windows: Edit JMETER_HOME/bin/jmeter.bat

# Find the line with 'set HEAP=' and change to:

set HEAP=-Xms1g -Xmx4g

# Linux/Mac: Edit JMETER_HOME/bin/jmeter.sh

# Find 'HEAP=' line and set:

HEAP="-Xms1g -Xmx4g"

# -Xms = Initial heap size (1 GB)

# -Xmx = Maximum heap size (4 GB)

# Recommended: Set -Xmx to 60-70% of available RAM
```

Method 2 – JVM_ARGS Environment Variable:

```
# Windows (PowerShell)

$env:JVM_ARGS = '-Xms2g -Xmx8g'

jmeter

# Linux/macOS

export JVM_ARGS='-Xms2g -Xmx8g'

jmeter
```

Memory Recommendations by Test Size:

Virtual Users	Recommended -Xmx	System RAM
Up to 500	2 GB	8 GB
500 – 2,000	4 GB	16 GB
2,000 – 5,000	8 GB	32 GB
5,000+	Distributed mode	64+ GB (multiple nodes)

■ **Warning:** Always run JMeter tests in **Non-GUI mode** for actual load testing. GUI mode consumes extra resources and skews results. Use: `jmeter -n -t testplan.jmx -l results.jtl`

SECTION 04

JMeter Recording

Proxy Setup • BlazeMeter • HAR Conversion • Playback & Validation

Introduction to JMeter Recording

Script recording captures real browser HTTP/HTTPS traffic and converts it into JMeter test elements automatically. Instead of manually writing every request, you simply navigate through the application and JMeter records every HTTP call.

Recording Approaches Comparison:

Method	How It Works	Best For	Limitations
JMeter Proxy	Browser routes traffic through JMeter HTTP Proxy Server	Full control, Proxy Setup	Requires SSL cert setup
BlazeMeter Recorder	Chrome extension captures HAR	Quick recording, Beginners	Requires BlazeMeter account
HAR to JMX	Export HAR from DevTools; convert using JMeter tools	Offline recording	No real-time feedback

Method 1: Proxy Setup & JMeter Recording

Step 1 – Configure JMeter HTTP Proxy Server:

- Right-click on **WorkBench** (or Test Plan) > Add > Non-Test Elements > **HTTP(S) Test Script Recorder**
- Set **Port: 8888** (or any free port)
- Set **Target Controller:** the Thread Group where requests will be saved
- Add URL patterns to **Include Patterns** (e.g., *.jsp, *.api.*)
- Add **Exclude Patterns** for static resources: *.css, *.js, *.png, *.gif

Step 2 – Configure Browser Proxy:

```
# Configure browser to use JMeter as proxy
```

```
# Firefox: Settings > Network Settings > Manual Proxy:
```

```
# HTTP Proxy: localhost Port: 8888
```

```
# Check: 'Also use this proxy for HTTPS'
```

```
# Chrome (via system proxy or extension FoxyProxy):
```

```
# HTTP: 127.0.0.1:8888
```

```
# HTTPS: 127.0.0.1:8888
```

```
# Command-line Chrome (launch with proxy):
```

```
# Windows:
```

```
chrome.exe --proxy-server='http://localhost:8888'
```

```
# Mac:
```

```
/Applications/Google\ Chrome.app/Contents/MacOS/Google\ Chrome \
```

```
--proxy-server='http://localhost:8888'
```

Step 3 – Install JMeter SSL Certificate in Browser:

- Start the JMeter Proxy Server once — it auto-generates **ApacheJMeterTemporaryRootCA.crt**
- **Firefox:** Settings > Privacy & Security > View Certificates > Import > Select .crt file
- **Chrome/Edge:** Settings > Privacy > Manage Certificates > Trusted Root CAs > Import
- Check 'Trust this CA to identify websites'

Step 4 – Record the Script:

```
JMeter Recording Workflow:
```

```
1. In JMeter: Click 'Start' on the HTTP(S) Test Script Recorder
```

```
2. In Browser: Navigate to application URL
```

```
3. Perform all user actions (login, search, checkout, logout)
```

```
4. In JMeter: Click 'Stop' when done
```

```
5. Recorded requests appear under the target Thread Group
```

```
Pro Tips:
```

```
- Add a Timer (e.g., Constant Timer 1000ms) to simulate think time
```

```
- Use 'Clear' on the proxy between recordings to avoid duplicates
```

```
- Group related requests using Transaction Controllers
```

```
- Add Response Assertions during recording for basic validation
```

Method 2: Script Recording using BlazeMeter

BlazeMeter's Chrome extension records browser activity and exports it directly as a JMeter JMX file, eliminating the need for manual proxy and certificate setup.

#	Action	Details
1	Install Extension	Search 'BlazeMeter' in Chrome Web Store and install the recorder extension
2	Sign In	Create a free BlazeMeter account at blazemeter.com and sign in via the extension
3	Start Recording	Click the BlazeMeter icon > Enter a test name > Click 'Record' button
4	Perform Actions	Navigate through your application — all HTTP(S) requests are captured automatically
5	Stop Recording	Click 'Stop' in the extension when the user journey is complete
6	Export as JMX	Click 'Edit' > Review captured requests > Click '.jmx' to download the JMeter script
7	Open in JMeter	File > Open > Select downloaded .jmx file to import into JMeter

■ **Tip:** BlazeMeter also lets you run the recorded script directly in the cloud without setting up infrastructure. Free tier supports up to 50 virtual users.

Method 3: HAR to JMX Conversion

A HAR (HTTP Archive) file is a JSON-formatted log of all browser network activity. You can export it from Chrome DevTools and convert it to a JMeter JMX script.

Capturing a HAR File:

Chrome Browser Steps:

1. Press F12 to open DevTools
2. Click the 'Network' tab
3. Click the red record button (if not already recording)
4. Perform all user actions on the application
5. Right-click anywhere in the Network requests list
6. Select: 'Save all as HAR with content'
7. Save the .har file to your local machine

Firefox: DevTools > Network > Gear icon > Save All As HAR

Converting HAR to JMX:

Option A: JMeter built-in (JMeter 5.5+)

Tools menu > Import from HAR > Select .har file > OK

Requests are created in the current Test Plan

Option B: Online converter

<https://www.blazemeter.com/blog/har-to-jmx>

Upload HAR > Download JMX

Option C: Command line (jmeter-har-importer plugin)

```
java -jar har-importer.jar --har myrecording.har --output script.jmx
```

After conversion:

- Review and clean up static resource requests
- Add correlation for dynamic values (session tokens, CSRF)

- Add assertions and timers

Playback and Validation of Request/Response

After recording, the script must be played back with a single virtual user (VU) and validated before scaling to load. This step catches correlation issues, hardcoded values, and request failures.

Playback Setup:

Thread Group settings for validation playback:

Number of Threads (users): 1

Ramp-Up Period: 1 second

Loop Count: 1

Add Listeners for validation:

- View Results Tree (see request/response details)

- Summary Report (see pass/fail counts)

- Response Time Graph (basic trend)

Run the script:

Press Ctrl+R or click the Green Play button

Validation Checklist:

- All requests return **HTTP 200** (or expected status codes — 201, 302, etc.)
- Response bodies contain expected text (use **Response Assertion**)
- Dynamic values are properly correlated (session IDs, CSRF tokens, auth tokens)
- Response times are within acceptable range for single user
- No hardcoded user-specific values (use **CSV Data Set Config** for parameterisation)
- Cookies are handled correctly (check **Cookie Manager** is present)
- Content-Type headers match expected values (JSON, HTML, XML)

Adding Response Assertions:

JMeter Response Assertion - Common Configurations:**1. Text Response Contains Expected String:**

Right-click Request > Add > Assertions > Response Assertion

Field to Test: Response Body

Pattern Matching: Contains

Pattern: 'Welcome, John' or 'order_id'

2. Response Code Assertion:

Field to Test: Response Code

Pattern: 200

3. JSON Path Assertion (for APIs):

Add > Assertions > JSON Assertion

JSON Path: \$.data.userId

Expected Value: 12345

4. Duration Assertion:

Add > Assertions > Duration Assertion

Duration in milliseconds: 3000 (must respond within 3s)

Common Correlation Scenarios:

Dynamic Value	JMeter Extractor	Example
Session Token (Cookie)	HTTP Cookie Manager (auto)	SESSIONID, .ASPXAUTH
CSRF Token (Form)	CSS/JQuery Extractor	input[name=_token]
Auth Token (JSON)	JSON Path Extractor	\$.access_token
ViewState (ASP.NET)	Regular Expression Extractor	__VIEWSTATE.*value="(.*?)"
Redirect URL	Regular Expression Extractor	location: (.*)\r\n

Activity: Record Complete User Journey & Validate

This end-to-end activity brings together all concepts: proxy setup, recording, correlation, parameterisation, assertions, and validation playback.

Scenario: E-Commerce User Journey

Step	Action	Detail
Step 1	Launch & Configure JMeter	Open JMeter, set Thread Group (1 user, 1 loop), add HTTP(S) Test Script Recorder on port
Step 2	Configure Browser Proxy	Set browser to use localhost:8888 as HTTP/HTTPS proxy
Step 3	Start Recording	Click 'Start' on JMeter recorder, open browser
Step 4	Navigate to Homepage	Go to application URL — JMeter captures the request
Step 5	User Login	Enter credentials and click Login — capture login POST request
Step 6	Search for Product	Type a product name in the search bar and press Enter
Step 7	View Product Detail	Click a product to open the detail page

Step 8	Add to Cart	Click 'Add to Cart' button
Step 9	View Cart	Navigate to the cart / basket page
Step 10	Checkout	Fill in shipping details and proceed to payment (use test card)
Step 11	Order Confirmation	Verify the order confirmation page loads
Step 12	Logout	Click Logout — capture the logout request
Step 13	Stop Recording	Click 'Stop' on JMeter recorder

Post-Recording Tasks:

- **Clean up:** Delete requests for static resources (.css, .js, .png, fonts)
- **Add Cookie Manager:** Right-click Thread Group > Add > Config > HTTP Cookie Manager
- **Add Header Manager:** Add Content-Type: application/json where needed for API calls
- **Correlate tokens:** Extract CSRF/auth tokens using JSON Path or RegEx Extractors; reference them in subsequent requests
- **Parameterise users:** Add CSV Data Set Config — use a users.csv with email, password columns
- **Add Assertions:** Verify homepage title, login success message, cart count, order ID
- **Add Think Time:** Add Constant Timer (1000–3000ms) between user actions to simulate real behaviour
- **Add Transaction Controllers:** Group Login, Search, Checkout into labelled transactions for reporting

Validation Playback:

1. Set Thread Group: 1 user, 1 loop, 1s ramp-up

2. Add 'View Results Tree' listener

3. Press Ctrl+R to run

4. Inspect each request:

- Green = Pass

- Red = Fail (investigate response body for errors)

5. Fix any failures:

- 302 Redirect: Ensure 'Follow Redirects' is checked

- 403 Forbidden: Check CSRF token extraction

- 401 Unauthorized: Verify session/auth token correlation

- 500 Server Error: Check request body format (JSON vs form data)

6. Re-run until ALL requests pass with HTTP 200 / expected codes

7. Validate response content matches expected values

8. Script is ready for load test execution

■ **Warning:** NEVER run load tests from the JMeter GUI. Always use non-GUI mode for accurate results and lower resource overhead: `jmeter -n -t UserJourney.jmx -l results.jtl -e -o ./report`

Keep Practising: Record > Correlate > Validate > Execute > Analyse